

World Wide Web (WWW)

Camadas de Transporte e Aplicação

Agenda

- Introdução
- Visão Geral da Arquitetura Web
- Páginas Estáticas e Dinâmicas
- *HyperText Transfer Protocol* (HTTP)
- Privacidade na Web

Introdução

World Wide Web (WWW)

- Consiste em uma vasta coleção de **páginas** (documentos) contendo **links** (endereços) para diversas páginas espalhadas pela Internet
- Principal fonte de informação no mundo atual
- Corresponde a um conjunto de clientes e servidores se comunicando através do protocolo HTTP

World Wide Web Consortium (W3C)

- Ajuda no desenvolvimento da web
- Padroniza seus protocolos
- Incentiva a interoperabilidade entre sites
- Endereço: www.w3.org

Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: Mosaic, primeiro navegador gráfico
- 1993 e 1994: Wandex, Excite e Yahoo (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: Cadê? (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)

Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: **Mosaic**, primeiro navegador gráfico
- 1993 e 1994: Wandex, Excite e Yahoo (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: Cadê? (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)



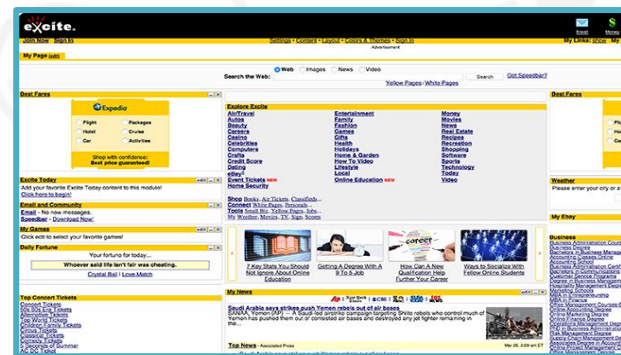
Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: Mosaic, primeiro navegador gráfico
- 1993 e 1994: **Wandex**, Excite e Yahoo (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: Cadê? (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)



Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: Mosaic, primeiro navegador gráfico
- 1993 e 1994: Wandex, **Excite** e Yahoo (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: Cadê? (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)



Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: Mosaic, primeiro navegador gráfico
- 1993 e 1994: Wandex, Excite e **Yahoo** (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: Cadê? (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)



Histórico da WWW

- 1989: Projeto inicial
- 1990: primeira versão (baseada em texto)
- 1993: Mosaic, primeiro navegador gráfico
- 1993 e 1994: Wandex, Excite e Yahoo (motores de busca)
- 1994: Amazon (livraria)
- 1995: Netscape (navegador)
- 1995: **Cadê?** (motor de busca na web brasileira)
- 1998: eBay (mercado de leilões)
- 1998: Google (buscador)
- 2004: Facebook (rede social)



Visão Geral da Arquitetura Web

Páginas Web

- Descritas em *HyperText Markup Language* (HTML)
- Visualizadas por um programa cliente: navegador (*browser*)
- Contêm *links* para outras páginas em qualquer lugar no mundo
- São estáticas ou dinâmicas
- Contêm objetos (e.g., imagens e vídeos) localizados ou não na mesma máquina

HyperText Markup Language (HTML)

- Linguagem de marcação de hipertexto
- Permite que os usuários produzam páginas estáticas ou dinâmicas
- Um exemplo de marcador é o `<b>` **que significa iniciar modo negrito**, e `</b>` seu término
- Páginas web modernas utilizam folhas de estilo (linguagem *Cascading Style Sheets*, CSS) para definir fontes, cores, tamanhos e outros atributos

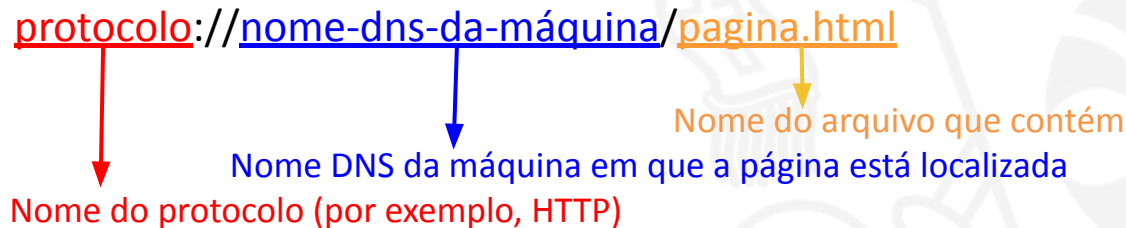
Links

- Endereços para outras páginas
- Aparência definida por folhas de estilo de tal forma que se parecem com qualquer coisa desejada pelo projetista do site
 - Aparência dinâmica, mudando quando, por exemplo, passamos o mouse sobre o *link* ou conforme data/horário/localização ou ...
 - No passado, eram sublinhados e com texto colorido (azul ou roxo)

Uniform Resource Locator (URL)

- Nome mundial de uma página
- Informa como buscar a página, onde ela está localizada e como acessá-la
- Dividida em três partes:

protocolo://nome-dns-da-máquina/pagina.html



The diagram illustrates the components of a URL. A red arrow points from 'protocolo' to 'Nome do protocolo (por exemplo, HTTP)'. A blue arrow points from 'nome-dns-da-máquina' to 'Nome DNS da máquina em que a página está localizada'. An orange arrow points from 'pagina.html' to 'Nome do arquivo que contém'.

Nome do protocolo (por exemplo, HTTP)

Nome DNS da máquina em que a página está localizada

Nome do arquivo que contém

Exemplos de URLs

Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Arquivo local	file:///usr/nathan/prog.c
mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins , about:telemetry e about:config

Exemplos de URLs



Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Arquivo local	file:///usr/nathan/prog.c
mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins , about:telemetry e about:config

Exemplos de URLs

Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
 ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Arquivo local	file:///usr/nathan/prog.c
mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins , about:telemetry e about:config

Exemplos de URLs

Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
 file	Arquivo local	file:///usr/nathan/prog.c
mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins , about:telemetry e about:config

Exemplos de URLs

Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Arquivo local	file:///usr/nathan/prog.c
 mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins, about:telemetry e about:config

Exemplos de URLs

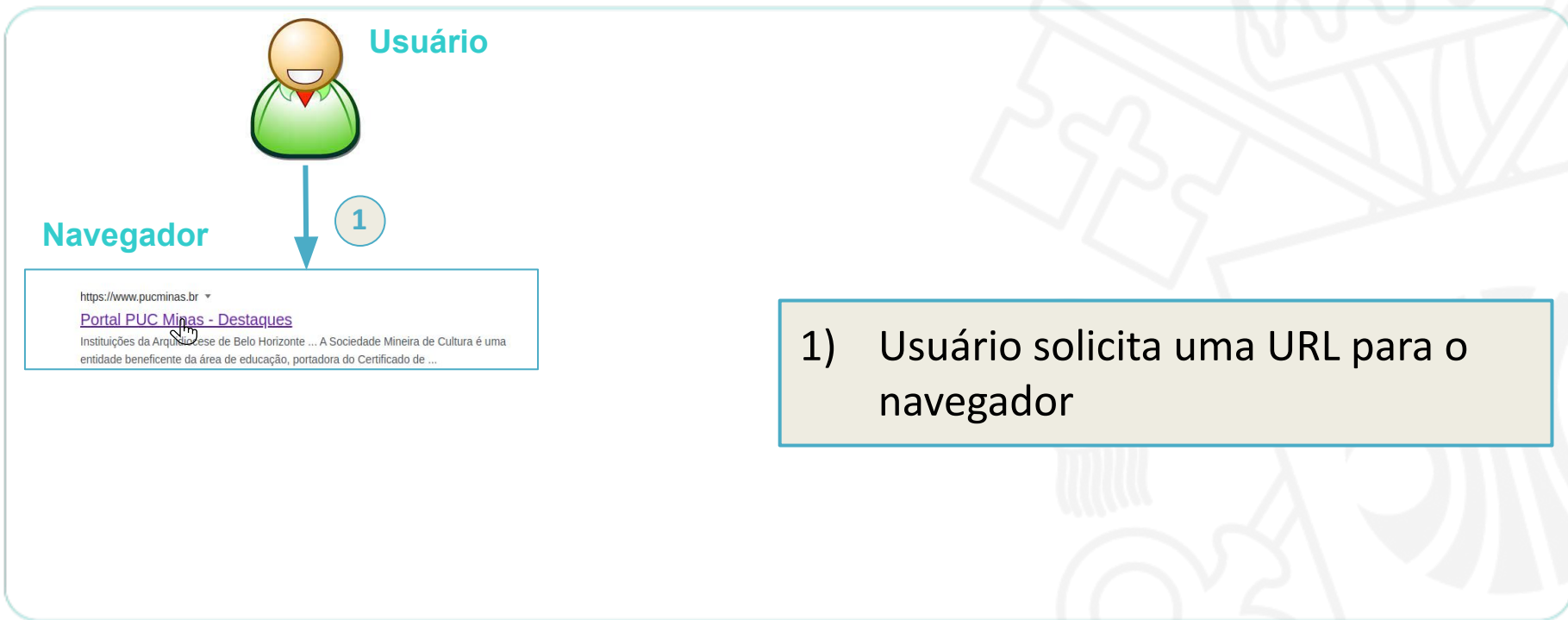
Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	
file	Arquivo local	
mailto	Envio de e-mail	
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
about	Informação do navegador	about:plugins, about:telemetry e about:config

RTSP e SIP estabelecem sessões de streaming de mídia e chamadas de áudio e vídeo, respectivamente

Exemplos de URLs

Nome	Usado para	Exemplo
http	Hipertexto (HTML)	http://www.pucminas.br
https	Hipertexto com segurança	https://www.pucminas.br
ftp	FTP	ftp://ftp.cs.vu.nl/pub/minix/README
file	Arquivo local	file:///usr/nathan/prog.c
mailto	Envio de e-mail	mailto:maxm@pucminas.br
rtsp	Streaming de mídia	rtsp://youtube.com/montypython.mpg
sip	Chamadas de multimídia	sip:eve@adversary.com
 about	Informação do navegador	about:plugins, about:telemetry e about:config

Acesso Passo-a-passo a um Link



Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

2

2) Navegador solicita endereço IP do servidor web ao de DNS (via UDP)

Acesso Passo-a-passo a um Link



Usuário



Servidor
DNS

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...

3

3) Servidor de DNS retorna o endereço IP solicitado

Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

4



Servidor
Web

4) Navegador faz uma conexão TCP com o servidor web

Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

5



Servidor
Web

5) Navegador faz solicitação HTTP de uma página para o servidor web

Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

6



Servidor
Web

6) Servidor web envia resposta HTTP com a página solicitada

Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

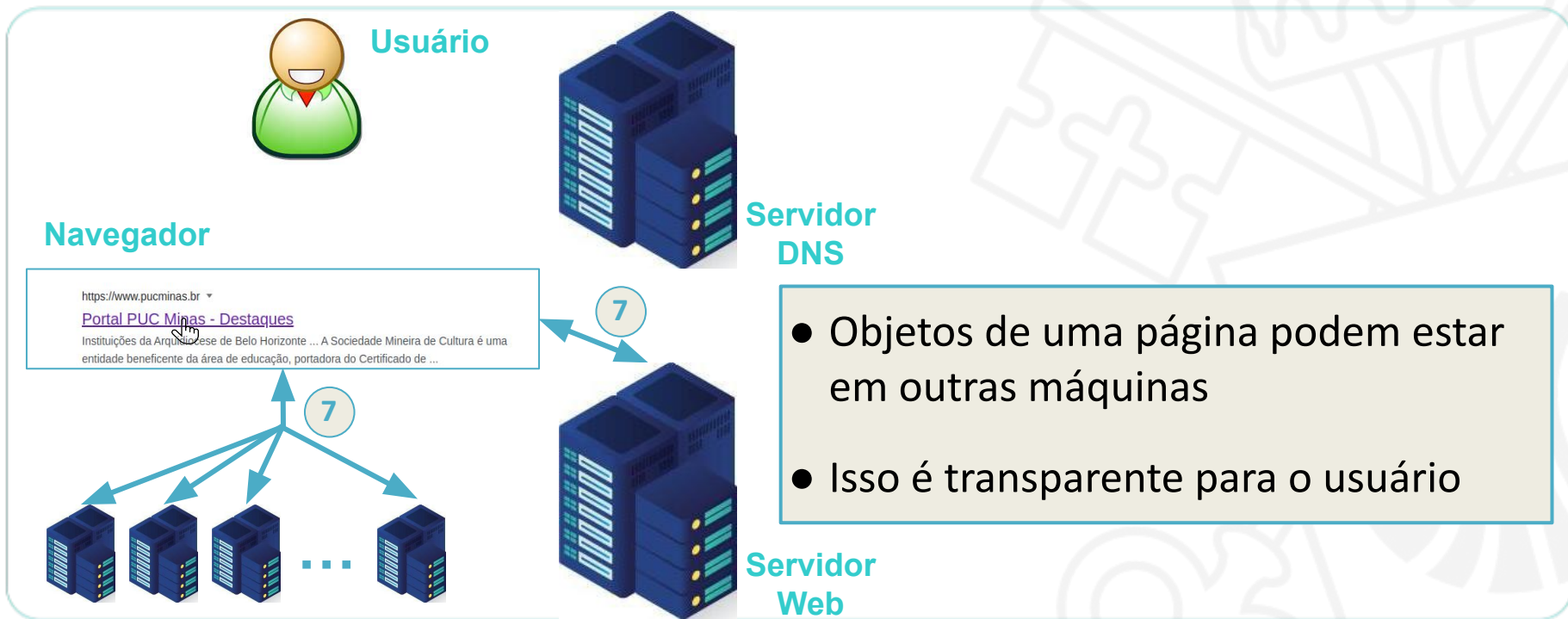
7



Servidor
Web

7) Navegador requisita outras páginas ao servidor web

Acesso Passo-a-passo a um Link



Acesso Passo-a-passo a um Link



*Pode acontecer em paralelo com a etapa anterior

Acesso Passo-a-passo a um Link



Usuário

Navegador

<https://www.pucminas.br>

[Portal PUC Minas - Destaques](#)

Instituições da Arquidiocese de Belo Horizonte ... A Sociedade Mineira de Cultura é uma entidade beneficente da área de educação, portadora do Certificado de ...



Servidor
DNS

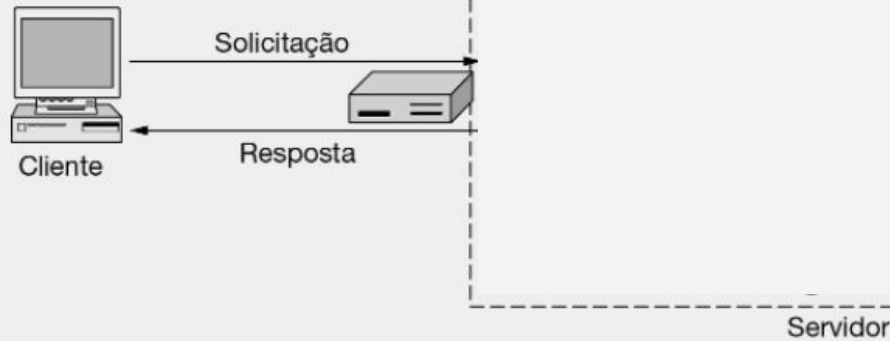
9



Servidor
Web

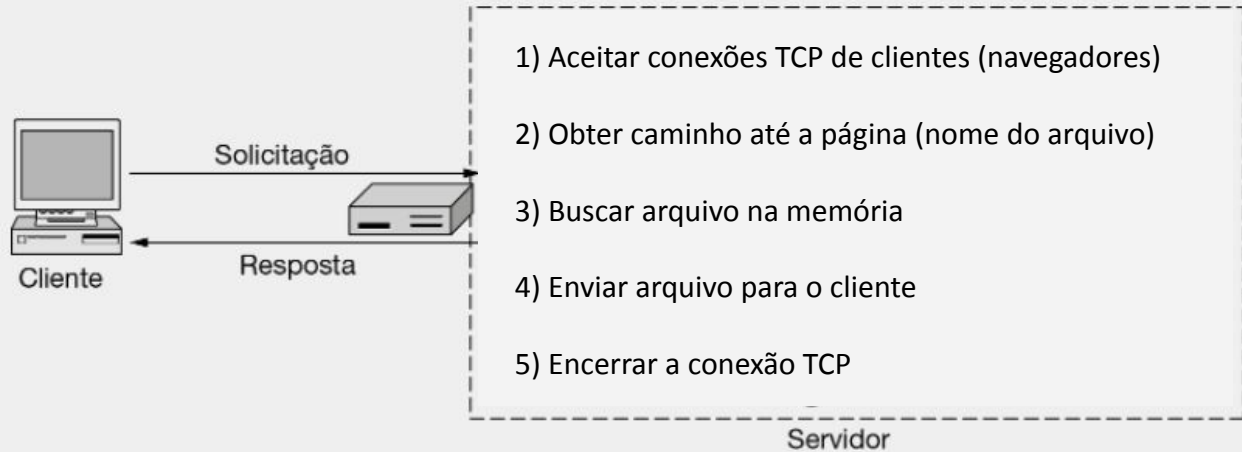
9) Navegador e servidor web terminam a conexão TCP

Tarefas do Servidor Web



FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

Tarefas do Servidor Web



FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

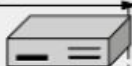
Tarefas do Servidor Web



Cliente

Solicitação

Resposta

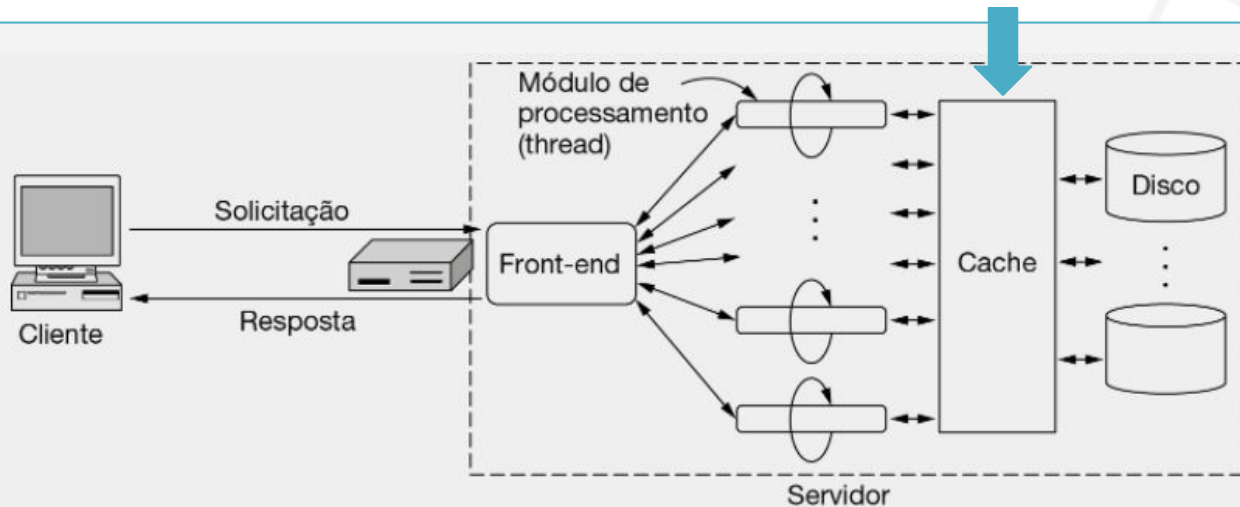


- 1) Aceitar conexões TCP de clientes (navegadores)
- 2) Obter caminho até a página (nome do arquivo)
- 3) Buscar arquivo na memória
- 4) Enviar arquivo para o cliente
- 5) Encerrar a conexão TCP

FONTE: A. Tanenbaum et al.

- Servidores atendem milhares de solicitações por segundo
- Busca na memória é lenta, logo, servidores usam:
 - *cache*
 - *multithread*
 - *proxy reverso*

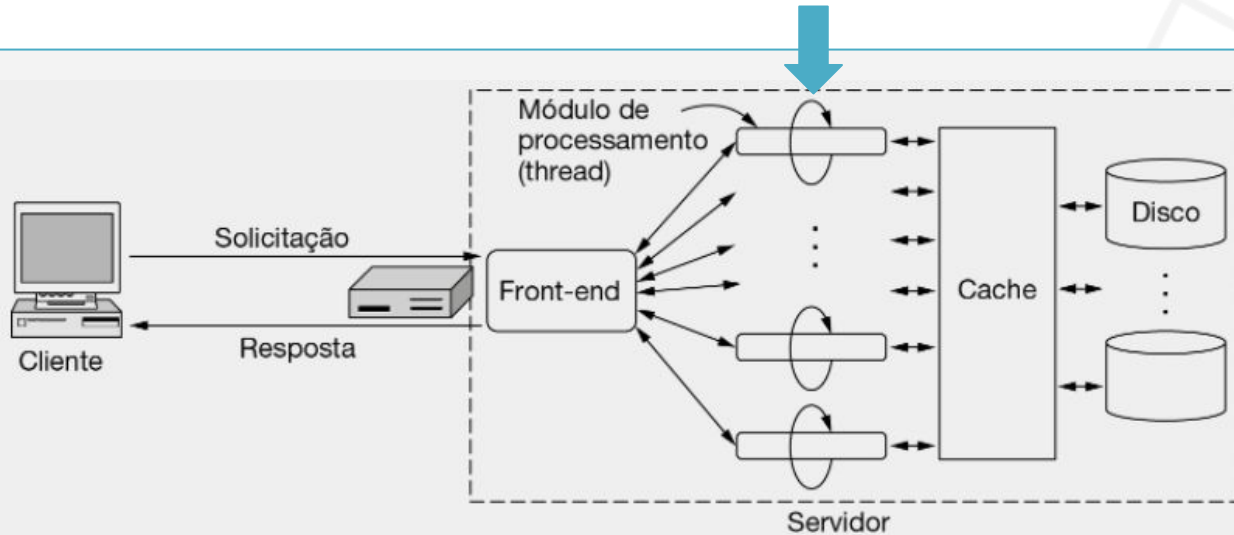
Tarefas do Servidor Web



Cache otimiza buscas recentes

FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

Tarefas do Servidor Web



Multithread permite que o processador adiante outras *threads* enquanto as anteriores aguardando a busca na memória

FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

Proxy Reverso

- São servidores de front-end* que buscam o conteúdo de outros servidores e enviam tais conteúdos (objetos) para os clientes
 - Ao carregar uma página, direcionamos o cliente para um *proxy reverso* que retorna objetos estáticos, carregando mais rapidamente parte do conteúdo da página
 - Enquanto isso, o *back-end* realiza operações complexas tais como busca na web ou em banco de dados ou a geração de conteúdo dinâmico
- Atuam em favor dos servidores, ao contrário de atuar em favor dos clientes

* Arquiteturas web modernas são divididas em *front-end* e *back-end*

Páginas Estáticas e Dinâmicas

Objetos Estáticos

- Existentes em quase todas as páginas web
- Exemplos: logotipo, folhas de estilo, cabeçalho, rodapé, imagem e vídeo
- Podem ser armazenados em *cache* (às vezes por muito tempo) e não precisam ser sistematicamente buscados

Aplicações Web

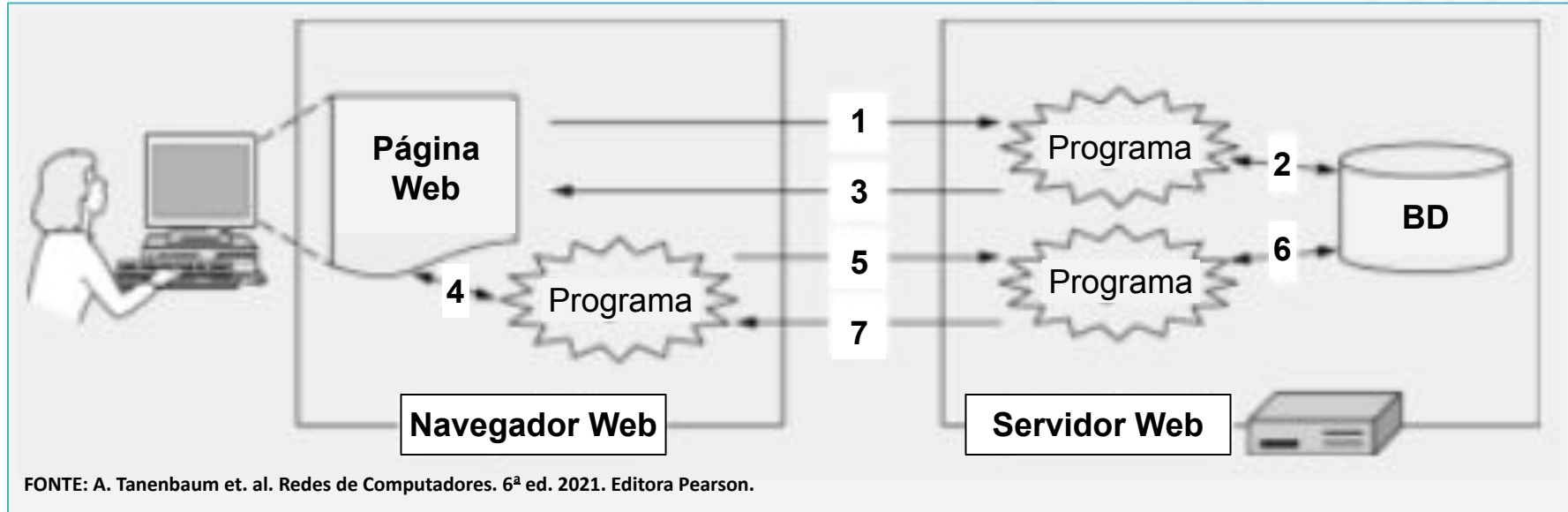
- Baseadas em páginas web dinâmicas
- São como as aplicações tradicionais, contudo, executadas no navegador
- Exemplos:
 - Sites de comércio eletrônico
 - Biblioteca
 - Exibição de mapas
 - Leitura e envio de e-mails
 - Colaboração em documentos
 - Serviços públicos
 - Serviços privados (e.g., médico, educação)

Aplicações Web

- Não precisa instalar programas
- Acessada a partir de diferentes computadores
- Modelo prevalente na computação em nuvem, mudando a computação dos *desktop* individuais para *clusters* de servidores na Internet

Conteúdo Dinâmico

- Gerado no servidor ou no cliente (navegador)



Hyper Text Transfer Protocol (HTTP)

Agenda

- Introdução
 - Visão Geral da Arquitetura Web
 - Páginas Estáticas e Dinâmicas
 - *HyperText Transfer Protocol (HTTP)*
 - Privacidade na Web
- Visão Geral
 - Métodos
 - Cabeçalhos de Mensagens
 - Melhorando Desempenho do HTTP
 - Versões do HTTP
 - HTTP vs. SMTP

Agenda

- Introdução
- Visão Geral da Arquitetura Web
- Páginas Estáticas e Dinâmicas
- *HyperText Transfer Protocol (HTTP)*
- Privacidade na Web

- **Visão Geral**
- Métodos
- Cabeçalhos de Mensagens
- Melhorando Desempenho do HTTP
- Versões do HTTP
- HTTP vs. SMTP

Hyper Text Transfer Protocol (HTTP)

- Protocolo simples do tipo solicitação-resposta para a transferência de páginas
- Especifica as mensagens enviadas pelos clientes e as respostas dos servidores
- Roda sobre TCP (pelo menos até o HTTP/3)
- Protocolo baseado em texto e que também utiliza o MIME
- Tem uma versão segura, o *Secure HTTP* (HTTPS) que utiliza o HTTP sobre um protocolo de transporte seguro (*Transport Layer Security*, TLS)

HTTP: Virando um Protocolo de Transporte

- Protocolo da camada de aplicação associado à web e que roda sobre o TCP
- Vem se tornando um “protocolo de transporte” ao permitir que processos remotos troquem dados sem um navegador. Por exemplo, via HTTP:

Exemplos usando HTTP no Transporte

- Player de mídia pode solicitar a um servidor informações sobre álbum
- Software antivírus pode baixar as atualizações mais recentes
- Desenvolvedores pode buscar arquivos do projeto
- Servidor de uma agência de viagens pode contactar o de uma locadora de veículos e reservar um carro para um pacote de férias vendido pela agência

Funcionamento Básico do HTTP

- Apresentado nos slides de acesso passo-a-passo a um *link*, onde:
 - 1) Servidor HTTP aguarda conexões (normalmente na porta 80)
 - 2) Cliente HTTP abre uma conexão TCP e envia uma mensagem de **requisição** para o servidor
 - 3) Servidor retorna uma mensagem de **resposta**, geralmente contendo o recurso requisitado
 - 4) Cliente e servidor fecham a conexão TCP

Agenda

- Introdução
- Visão Geral da Arquitetura Web
- Páginas Estáticas e Dinâmicas
- *HyperText Transfer Protocol (HTTP)*
- Privacidade na Web

- Visão Geral
- **Métodos**
- Cabeçalhos de Mensagens
- Melhorando Desempenho do HTTP
- Versões do HTTP
- HTTP vs. SMTP

Métodos de Requisição do HTTP

- Indicam a ação a ser executada em uma requisição do cliente HTTP
- Efetuados por um navegador web (poderia ser efetuado por outra aplicação)
- Principais métodos de requisição: ***GET*** e ***POST***

Métodos GET e POST

- **GET** solicita ao servidor que envie um arquivo (página)
- **POST** acrescenta algo a uma página web (e.g., envia dados para o servidor)
 - Busca uma página e faz o *upload* de dados para o servidor
 - URL requisitada não é um arquivo a ser recuperado e, sim, um programa para tratar os dados enviados pelo cliente
 - Normalmente, sua resposta é a saída de um programa e, não, um arquivo estático

Exemplos Práticos do GET e POST

- Suponha um sistema simples para controle de estoque

Controle de Estoque

Inserir Produto

Descrição: leite, pão, ...

Preço: 0.0

Quantidade: 0

Data de fabricação: 2022-05-25T18:45:56

Data de validade: 2022-11-25

Inserir

Relação de Produtos

ID	Descrição	Preço	Detalhar	Atualizar	Excluir
1	Chocolate	10.0			
4	Hamburger	23.0			
17	Feijão	23.0			
38	Pão	12.0			
39	Leite	20.0			

Exemplo Prático do Método GET

- No navegador, o usuário acessa a opção abaixo de **Detalhar Produto**

Controle de Estoque

Inserir Produto

Descrição:

Preço:

Quantidade:

Data de fabricação:

Data de validade:

Relação de Produtos

ID	Descrição	Preço	Detalhar	Atualizar	Excluir
1	Chocolate	10.0			
4	Hamburger	23.0			
17	Feijão	23.0			
38	Pão	12.0			
39	Leite	20.0			

Exemplo Prático do Método GET

- No navegador, o usuário acessa a opção abaixo de **Detalhar Produto**

```
<tr bgcolor="#fff5dd">
  <td>1</td>
  <td>Chocolate</td>
  <td>10.0</td>
  <td align="center" valign="middle"><a href="/produto/1"></a></td>
  <td align="center" valign="middle"><a href="/produto/update/1"></a>
  <td align="center" valign="middle"><a href="javascript:confirmarDeleteProduto('1', 'Chocolate', '10.0');">
  <td>4</td>
  <td>Hamburger</td>
  <td>23.0</td>
  <td align="center" valign="middle"><a href="/produto/4"></a></td>
  <td align="center" valign="middle"><a href="/produto/update/4"></a>
  <td align="center" valign="middle"><a href="javascript:confirmarDeleteProduto('4', 'Hamburger', '23.0');"> |  |  |
| 4  | Hamburger | 23.0  |  |  |  |
| 17 | Feijão    | 23.0  |  |  |  |
| 38 | Pão       | 12.0  |  |  |  |
| 39 | Leite     | 20.0  |  |  |  |

## Exemplo Prático do Método GET

- No navegador, o usuário acessa a opção abaixo de **Detalhar Produto**

```
<tr bgcolor="#fff5dd">
 <td>1</td>
 <td>Chocolate</td>
 <td>10.0</td>
 <td align="center" valign="middle"></td>
 <td align="center" valign="middle"></td>
 <td align="center" valign="middle"></td>
</tr>

<tr bgcolor="#dddddd">
 <td>4</td>
 <td>Hamburger</td>
 <td>23.0</td>
 <td align="center" valign="middle"></td>
 <td align="center" valign="middle"></td>
 <td align="center" valign="middle"></td>
</tr>
```

Relação de Produtos

ID	Descrição	Preço	Detalhar	Atualizar	Excluir
1	Chocolate	10.0			
4	Hamburger	23.0			
17	Feijão	23.0			
38	Pão	12.0			
39	Leite	20.0			

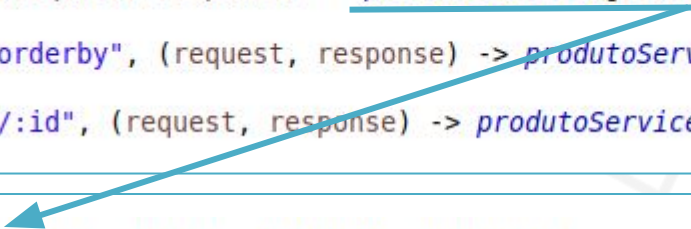
Solicitamos o recurso “**produto/1**” disponível no servidor web, onde 1 é o identificador (ID) do produto desejado

## Exemplo Prático do Método GET

- Servidor web tem uma opção para tratar a requisição **GET *produto/:id***



```
get("/produto/:id", (request, response) -> produtoService.get(request, response));
get("/produto/list/:orderby", (request, response) -> produtoService.getAll(request, response));
get("/produto/update/:id", (request, response) -> produtoService.getToUpdate(request, response));
```



```
public Object get(Request request, Response response) {
 int id = Integer.parseInt(request.params(":id"));
 Produto produto = (Produto) produtoDAO.get(id);

 if (produto != null) {
 response.status(200); // success
 makeForm(FORM_DETAIL, produto, FORM_ORDERBY_DESCRICA0);
 } else {
 response.status(404); // 404 Not found
 String resp = "Produto " + id + " não encontrado.";
 makeForm();
 }
 return form;
}
```

- # Método GET
- ## para o navegador
- 0.0

Quantidade: 8
- Validade: 2023-04-04

<b>Detalhar Produto (ID 1)</b>		
Descrição: Chocolate	Preço: 10.0	Quantidade: 8
Data de fabricação: 2020-04-04T00:00	Data de validade: 2023-04-04	

Quantidade: 8

Data de validade: 2023-04-04

[illegible]

# Exemplos Práticos do GET e POST

- Voltando ao nosso sistema simples para controle de estoque

## Controle de Estoque

### Inserir Produto

Descrição: leite, pão, ...

Preço: 0.0

Quantidade: 0

Data de fabricação: 2022-05-25T18:45:56

Data de validade: 2022-11-25

Inserir

### Relação de Produtos

ID	Descrição	Preço	Detalhar	Atualizar	Excluir
1	Chocolate	10.0			
4	Hamburger	23.0			
17	Feijão	23.0			
38	Pão	12.0			
39	Leite	20.0			

# Exemplo Prático do Método POST

- No navegador, temos a opção abaixo de Inserir Produto

## Controle de Estoque

### Inserir Produto

Descrição: leite, pão, ...

Preço: 0.0

Quantidade: 0

Data de fabricação: 2022-05-25T18:45:56

Data de validade: 2022-11-25

Inserir

### Relação de Produtos

ID	Descrição	Preço	Detalhar	Atualizar	Excluir
1	Chocolate	10.0			
4	Hamburger	23.0			
17	Feijão	23.0			
38	Pão	12.0			
39	Leite	20.0			





- No navegador, temos a opção abaixo de **Inserir Produto**

1) Usuário preenche o formulário e pressiona o botão de Inserir

2) Navegador faz uma requisição **POST** com ação **/produto/insert** para o servidor

Inserir Produto

Descrição: leite, pão, ...

Preço: 0.0

Quantidade: 0

Data de fabricação: 2022-05-25T18:45:56

Data de validade: 2022-11-25

Inserir

```
<form class="form--register" action="/produto/insert" method="post" id="form-add">
 <table width="80%" bgcolor="#f3f3f3" align="center">
 <tr>
 <td colspan="3" align="left"> &Inserir Produto</td>
 </tr>
 <tr>
 <td colspan="3" align="left"> & </td>
 </tr>
 <tr>
 <td> &Descrição:< input class="input--register" type="text" name="descricao" value="leite, pão, ..."></td>
 <td>Preço:< input class="input--register" type="text" name="preco" value="0.0"></td>
 <td>Quantidade:< input class="input--register" type="text" name="quantidade" value="0"></td>
 </tr>
 <tr>
 <td> &Data de fabricação:< input class="input--register" type="text" name="dataFabricacao" value="2022-05-24T12:41:35"></td>
 <td>Data de validade:< input class="input--register" type="text" name="dataValidade" value="2022-11-24"></td>
 <td align="center"><input type="submit" value="Inserir" class="input--main__style input--button"></td>
 </tr>
 </table>
</form>
```

Idade	Atualizar	Excluir
1		
2		
3		
4		
5		

## Exemplo Prático do Método POST

- Servidor web tem uma opção para tratar a requisição **POST** *produto/insert*



```
post("/produto/insert", (request, response) -> produtoService.insert(request, response));
```



```
public Object insert(Request request, Response response) {
 String descricao = request.queryParams("descricao");
 float preco = Float.parseFloat(request.queryParams("preco"));
 int quantidade = Integer.parseInt(request.queryParams("quantidade"));
 LocalDateTime dataFabricacao = LocalDateTime.parse(request.queryParams("dataFabricacao"));
 LocalDate dataValidade = LocalDate.parse(request.queryParams("dataValidade"));

 String resp = "";

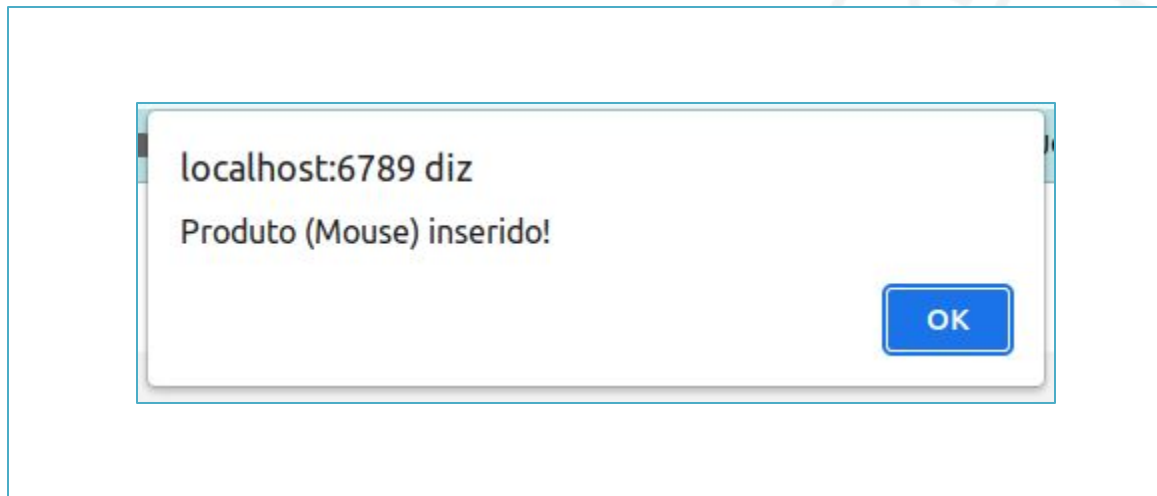
 Produto produto = new Produto(-1, descricao, preco, quantidade, dataFabricacao, dataValidade);

 if(produtodao.insert(produto) == true) {
 resp = "Produto (" + descricao + ") inserido!";
 response.status(201); // 201 Created
 } else {
 resp = "Produto (" + descricao + ") não inserido!";
 response.status(404); // 404 Not found
 }
}
```



## Exemplo Prático do Método POST

- Servidor web retorna resultado da a requisição **POST *produto/insert*** e o navegador pode gerar uma mensagem para o usuário



# Observação sobre o Método GET

- GET também pode enviar pequenas quantidades de dados para o servidor
- Dados codificados na URL após o sinal de “?”
  - Por exemplo, [www.xpto.com/page.html?cor=verde&id=5](http://www.xpto.com/page.html?cor=verde&id=5)

# Outros Métodos de Requisição do HTTP

Método	Descrição
<b>HEAD</b>	Solicita a leitura do cabeçalho de uma página <sup>(1)</sup>
<b>PUT</b>	Solicita o armazenamento de uma página <sup>(2)</sup>
<b>DELETE</b>	Remove uma página <sup>(2)</sup>
<b>TRACE</b>	Ecoa uma solicitação recebida
<b>CONNECT</b>	Conecta através de um <i>proxy</i>
<b>OPTIONS</b>	Consulta opções para uma página

<sup>(1)</sup> Usado para obter informações para indexação ou testar a validade/disponibilidade de um URL

<sup>(2)</sup> Autenticação e permissão são requisitos

# Mensagens de Resposta do HTTP

Código	Significado	Exemplos
1XX	Informação	100 = o servidor concorda em atender a requisição do cliente
2XX	Sucesso	200 = requisição bem sucedida; 204 = sem conteúdo
3XX	Redirecionamento	301 = página movida; 304 = página em <i>cache</i> ainda é válida
4XX	Erro do cliente	403 = página proibida; 404 = página não localizada
5XX	Erro do servidor	500 = erro interno no servidor; 503 = tente de novo mais tarde

# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - **Cabeçalhos de Mensagens**
  - Melhorando Desempenho do HTTP
  - Versões do HTTP
  - HTTP vs. SMTP

# Mensagens de Requisição / Resposta

- Compostas por uma ou mais linhas de texto ASCII
- A primeira linha é a de solicitação, em seguida, temos linhas de cabeçalho e o corpo da mensagem

<linha de solicitação>

<linhas de cabeçalho>

*Header 1: value*  
*Header 2: value*  
*...*  
*Header n: value*

<corpo da mensagem>

# Mensagens de Requisição / Resposta

- Compostas por uma ou mais linhas de texto ASCII
- A primeira linha é a de solicitação, em seguida, temos linhas de cabeçalho e o corpo da mensagem

<linha de solicitação>

<linhas de cabeçalho>

*Header 1: value*

*Header 2: value*

*...*

*Header n: value*

<corpo da mensagem>

# Linha de Solicitação nas Requisições

<método>\* <caminho recurso requisitado> <versão do HTTP>

Exemplo: GET /path/to/file/index.html HTTP/1.1

\* Nome do método é maiúsculo e *case sensitive* (por exemplo, get e Post estariam inválidos)



# Linha de Solicitação nas Respostas

<versão do HTTP> <código do status da resposta>

Exemplo (1): HTTP/1.1 200 OK

Exemplo (2): HTTP/1.1 404 Not Found

# Mensagens de Requisição / Resposta

- Compostas por uma ou mais linhas de texto ASCII
- A primeira linha é a de solicitação, em seguida, temos linhas de cabeçalho e o corpo da mensagem

<linha de solicitação>

<linhas de cabeçalho>

*Header 1: value*  
*Header 2: value*  
*...*  
*Header n: value*

<corpo da mensagem>

# Linhas de Cabeçalho

- Permitem que cliente ou servidor passe informações adicionais nas requisições ou respostas HTTP
- Cabeçalhos de requisição são da forma <nome:valor>, sendo que o nome é *case-insensitive*
- HTTP 1.0 define 16 cabeçalhos; e o HTTP 1.1, 46 cabeçalhos
- Alguns cabeçalhos são válidos nas mensagens de requisição, de resposta ou em ambos

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres aceitáveis para o cliente
<i>Accept-Encoding</i>	Codificações de páginas que o cliente pode manipular
<i>Accept-Language</i>	Idiomas com os quais o cliente pode lidar
<i>If-Modified-Since</i>	Data e hora para verificar atualização
<i>If-None-Match</i>	Tags enviadas anteriormente para verificar atualização
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do qual a solicitação veio
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres
<i>Accept-Encoding</i>	Codificações de compressão
<i>Accept-Language</i>	Idiomas com os quais o cliente pode se comunicar
<i>If-Modified-Since</i>	Data e hora para verificar atualização
<i>If-None-Match</i>	Tags enviadas anteriormente para verificar atualização
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do qual a solicitação veio
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

Cliente informa seu navegador (e.g, Chrome/74.0.3729.169 e Mozilla/5.0), permitindo que o servidor ajuste sua resposta

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres
<i>Accept-Encoding</i>	Codificações de páginas
<i>Accept-Language</i>	Idiomas com os quais o cliente pode se comunicar
<i>If-Modified-Since</i>	Data e hora para a qual o cliente possui uma cópia atualizada
<i>If-None-Match</i>	Tags enviadas anteriormente
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais de autenticação
<i>Referrer</i>	URL anterior do cliente
<i>Cookie</i>	Cookie previamente enviado

Cliente informa o que pode aceitar supondo um repertório limitado daquilo que é aceitável

*Accept*: tipos MIME (e.g., text/html)

*Accept-Charset*: conjunto de caracteres (e.g., ISO-8859-1)

*Accept-Encoding*: métodos de compressão (e.g., gzip)

*Accept-Language*: idioma natural (e.g., português)

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres aceitáveis para o cliente
<i>Accept-Encoding</i>	Codificações de páginas que o cliente pode manipular
<i>Accept-Language</i>	Idiomas com os quais o cliente pode lidar
<i>If-Modified-Since</i>	Data e hora para verificar atualização
<i>If-None-Match</i>	Tags enviadas anteriormente para verificar atualização
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais de autenticação
<i>Referrer</i>	URL anterior do cliente
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

Usados no *caching*, permite que o cliente solicite o envio de uma se a cópia em *cache* estiver desatualizada

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres aceitáveis para o cliente
<i>Accept-Encoding</i>	Codificações de páginas que o cliente pode manipular
<i>Accept-Language</i>	Idiomas com os quais o cliente pode lidar
<i>If-Modified-Since</i>	Data e hora para verificar atualização
<i>If-None-Match</i>	Tags enviadas anteriormente para verificar atualização
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do cliente
<i>Cookie</i>	Cookie previamente enviado

Cabeçalho obrigatório e obtido na URL

Identifica o servidor, pois alguns IPs têm múltiplos nomes DNS



# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres aceitáveis para o cliente
<i>Accept-Encoding</i>	Codificações de páginas que o cliente pode manipular
<i>Accept-Language</i>	Idiomas com os quais o cliente pode se comunicar
<i>If-Modified-Since</i>	Data e hora para a qual o conteúdo deve ser mais recente
<i>If-None-Match</i>	Tags enviadas anteriormente para o conteúdo
<i>Host</i>	Nome DNS do servidor
<i>Authorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do qual a solicitação veio
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

Necessário para páginas protegidas em que o cliente pode precisar de apresentar seu direito de acessar a página solicitada

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas o cliente pode aceitar
<i>Accept-Charset</i>	Conjuntos de caracteres suportados pelo cliente
<i>Accept-Encoding</i>	Codificações de compressão suportadas pelo cliente
<i>Accept-Language</i>	Idiomas com os quais o cliente pode lidar
<i>If-Modified-Since</i>	Data e hora para verificar atualização
<i>If-None-Match</i>	Tags enviadas anteriormente para verificar atualização
<i>Host</i>	Nome DNS do servidor
<i>Autorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do qual a solicitação veio
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

Útil para acompanhar a navegação na Web, pois informa aos servidores como um cliente chegou até a página

# Alguns Cabeçalhos de Requisição

Cabeçalho	Conteúdo
<i>User-Agent</i>	Informações sobre o navegador e sua plataforma
<i>Accept</i>	Tipo de páginas que o cliente pode manipular
<i>Accept-Charset</i>	Conjuntos de caracteres aceitáveis para o cliente
<i>Accept-Encoding</i>	Codificações de p
<i>Accept-Language</i>	Idiomas com os c
<i>If-Modified-Since</i>	Data e hora para
<i>If-None-Match</i>	Tags enviadas an
<i>Host</i>	Nome DNS do se
<i>Autorization</i>	Uma lista das credenciais do cliente
<i>Referrer</i>	URL anterior do qual a solicitação veio
<i>Cookie</i>	Cookie previamente definido, enviado de volta ao servidor

**Cookie** é um arquivo pequeno que o servidor grava no cliente para manter informações posteriormente úteis

Por exemplo, um site de comércio eletrônico pode usar cookies do lado do cliente para rastrear o que o cliente pediu até o momento

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo está codificado (por exemplo, gzip)
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página em bytes
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte do conteúdo da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de quando a página deixa de ser válida
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo é codificado
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página em bytes
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte do conteúdo da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de quando a página deixa de ser válida
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

Modo como o servidor envia cookies para o cliente

Servidor considera que o cliente salva os cookies para que sejam aproveitados em requisições futuras usando o cabeçalho Cookie

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo está codificado (por exemplo, gzip)
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de quando a página deixa de ser válida
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

Permite que o servidor envie informações dele para o cliente (e.g., versão de software)

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo está codificado (por exemplo, gzip)
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página em bytes
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte do conteúdo da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de quando a página deixa de ser válida
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceita

Permitem que o servidor descreva propriedades da página que está enviando

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo está codificado (por exemplo, gzip)
<i>Content-Language</i>	Idioma usado na
<i>Content-Length</i>	Tamanho da página
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte do conteúdo da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de quando a página deixa de ser válida
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

Cabeçalhos necessários para *caching*



# Alguns Cabeçalhos de Resposta

Cabeçalho	
<i>Set-Cookie</i>	Cookie para o cliente
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo foi codificado
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página em bytes
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte da página
<i>Last-Modified</i>	Data e hora da última modificação
<i>Expires</i>	Data e hora de expiração do cache
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

Servidor pede para o cliente tentar outra URL, por exemplo, devido a:

- Mudança na localização de uma página
- Existência de réplicas da página em outros servidores
- Temos uma página principal, mas redirecionamos os clientes para páginas regionais conforme seus endereços IP ou idiomas
- Temos uma página grande e um cliente prefere recebê-la em partes que estão disponíveis em servidores distintos

# Alguns Cabeçalhos de Resposta

Cabeçalho	Conteúdo
<i>Set-Cookie</i>	Cookie para o cliente armazenar
<i>Server</i>	Informações sobre o servidor
<i>Content-Encoding</i>	Como o conteúdo está codificado (por exemplo, gzip)
<i>Content-Language</i>	Idioma usado na página
<i>Content-Length</i>	Tamanho da página em bytes
<i>Content-Type</i>	Tipo MIME da página
<i>Content-Range</i>	Identifica uma parte do conteúdo da página
<i>Last-Modified</i>	Data e hora da última modificação da página
<i>Expires</i>	Data e hora de expiração
<i>Location</i>	Informa para onde o cliente deve enviar sua solicitação
<i>Accept-Ranges</i>	Servidor aceitará solicitações de intervalos de bytes

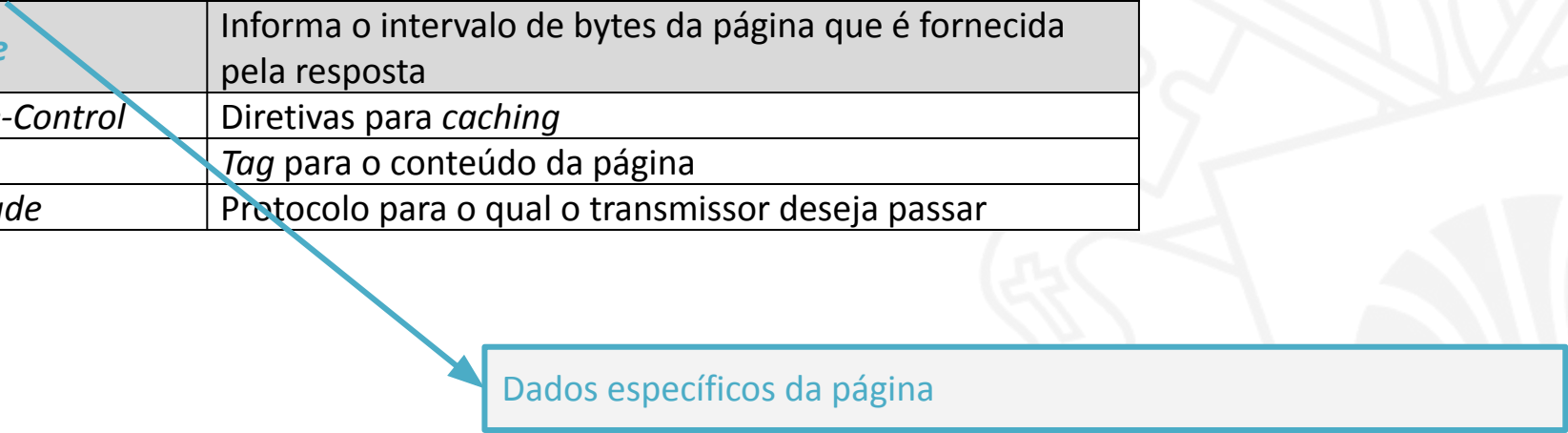
Informa que o servidor pode tratar solicitações de páginas parciais

# Alguns Cabeçalhos de Requisição/Resposta

Cabeçalho	Conteúdo
<i>Date</i>	Data e hora em que a mensagem foi enviada
<i>Range</i>	Informa o intervalo de bytes da página que é fornecida pela resposta
<i>Cache-Control</i>	Diretivas para <i>caching</i>
<i>ETag</i>	<i>Tag</i> para o conteúdo da página
<i>Upgrade</i>	Protocolo para o qual o transmissor deseja passar

# Alguns Cabeçalhos de Requisição/Resposta

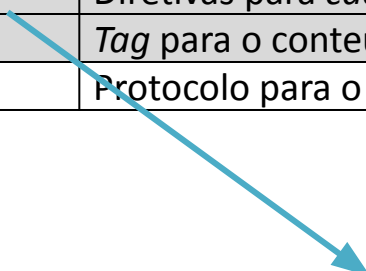
Cabeçalho	Conteúdo
<i>Date</i>	Data e hora em que a mensagem foi enviada
<i>Range</i>	Informa o intervalo de bytes da página que é fornecida pela resposta
<i>Cache-Control</i>	Diretivas para <i>caching</i>
<i>ETag</i>	<i>Tag</i> para o conteúdo da página
<i>Upgrade</i>	Protocolo para o qual o transmissor deseja passar



Dados específicos da página

# Alguns Cabeçalhos de Requisição/Resposta

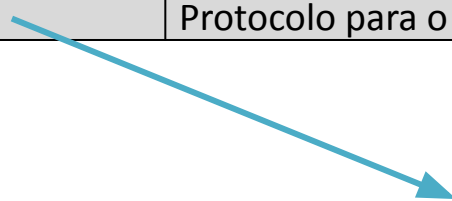
Cabeçalho	Conteúdo
<i>Date</i>	Data e hora em que a mensagem foi enviada
<i>Range</i>	Informa o intervalo de bytes da página que é fornecida pela resposta
<i>Cache-Control</i>	Diretivas para <i>caching</i>
<i>ETag</i>	<i>Tag</i> para o conteúdo da página
<i>Upgrade</i>	Protocolo para o qual o transmissor deseja passar



Cabeçalhos para *caching*

# Alguns Cabeçalhos de Requisição/Resposta

Cabeçalho	Conteúdo
<i>Date</i>	Data e hora em que a mensagem foi enviada
<i>Range</i>	Informa o intervalo de bytes da página que é fornecida pela resposta
<i>Cache-Control</i>	Diretivas para <i>caching</i>
<i>ETag</i>	<i>Tag</i> para o conteúdo da página
<i>Upgrade</i>	Protocolo para o qual o transmissor deseja passar



Migração para um novo protocolo de comunicação (e.g., futuro HTTP ou um transporte seguro)

Cliente informa o que aceita e o servidor, o que está usando

# Mensagens de Requisição / Resposta

- Compostas por uma ou mais linhas de texto ASCII
- A primeira linha é a de solicitação, em seguida, temos linhas de cabeçalho e o corpo da mensagem

<linha de solicitação>

<linhas de cabeçalho>

*Header 1: value*

*Header 2: value*

*...*

*Header n: value*

<corpo da mensagem>

# Corpo da Mensagem

- **Nas mensagens de requisição** é local que contém eventuais dados fornecidos pelo usuário a serem enviados para o servidor
- **Nas mensagens de resposta** é local onde o recurso requisitado se encontra
- Como o HTTP é baseado em texto, o corpo da mensagem pode ser codificado em MIME. Nesse caso, o cabeçalho Content-Type é MIME



# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - Melhorando Desempenho do HTTP
  - Versões
    - Caching
    - Replicação de Servidores
    - Redes de Entrega de Conteúdo

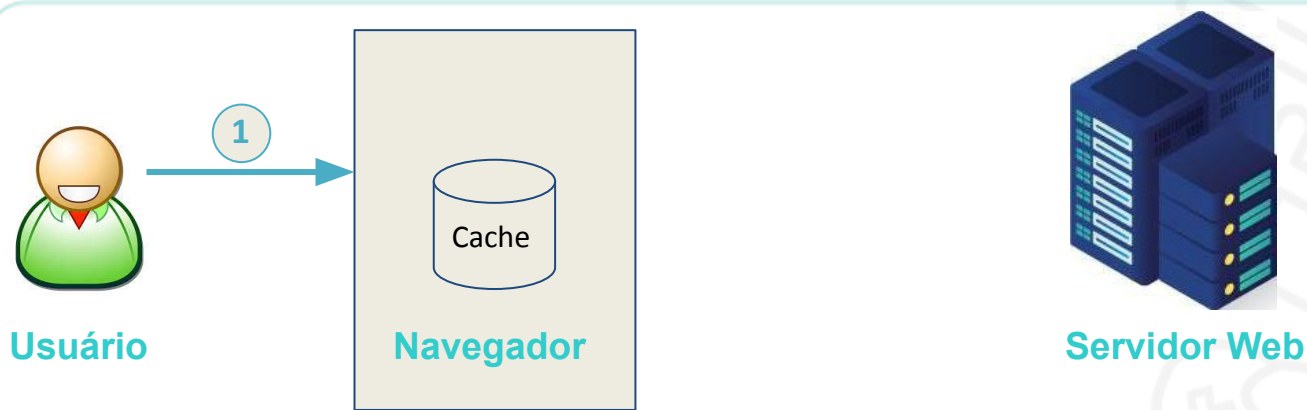
# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - Melhorando Desempenho do HTTP
  - Versões
    - *Caching*
    - Replicação de Servidores
    - Redes de Entrega de Conteúdo

# Introdução ao Web *Caching*

- Usuários normalmente acessam páginas recentemente visitadas
- Páginas têm recursos estáticos (e.g., imagens, folhas de estilo e *scripts*)
- É um desperdício baixar tais recursos sempre que uma página é requisitada
- **Web Caching**: guardar páginas (e seus recursos) para uso subsequente. Quando desejamos um recurso disponível na *cache*, ele não será baixado novamente

# Funcionamento Básico do Web *Caching*

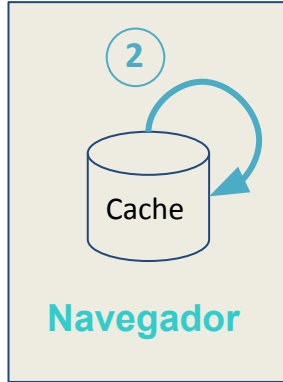


1) Usuário solicita página para o navegador

# Funcionamento Básico do Web *Caching*



Usuário



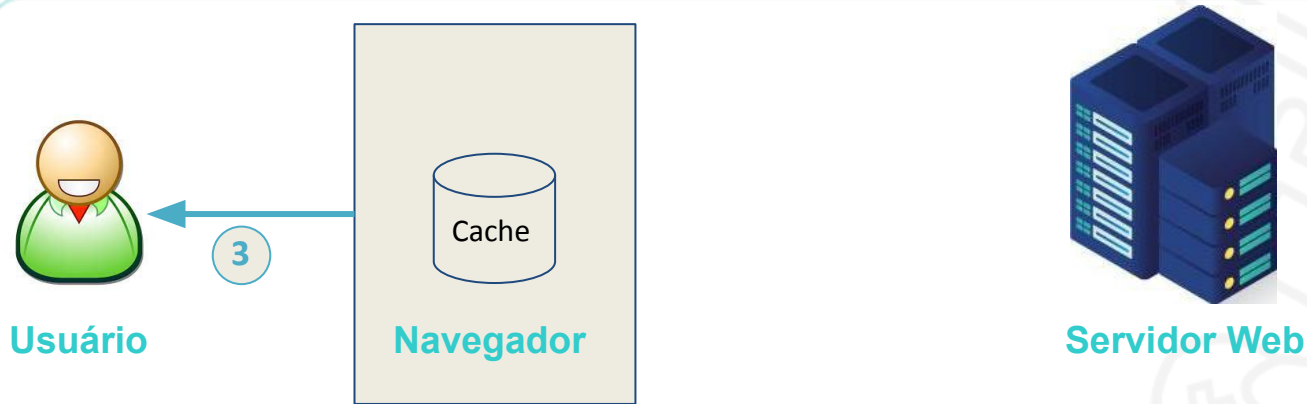
Navegador



Servidor Web

2) Navegador verifica se existe a página na *cache*

# Funcionamento Básico do Web *Caching*

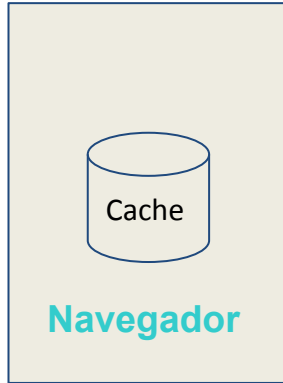


3) Se verdadeiro, o navegador retorna a página para o usuário sem acessar o servidor web nem gastar recursos de rede

# Funcionamento Básico do Web *Caching*



Usuário



4



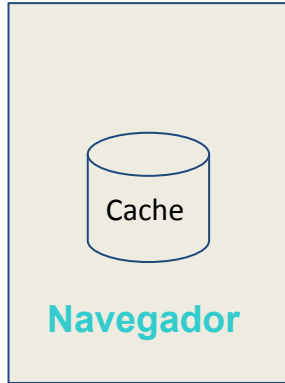
Servidor Web

4) Se não existir a página na *cache*, o navegador solicita a mesma para o servidor

# Funcionamento Básico do Web *Caching*



Usuário



5



Servidor Web

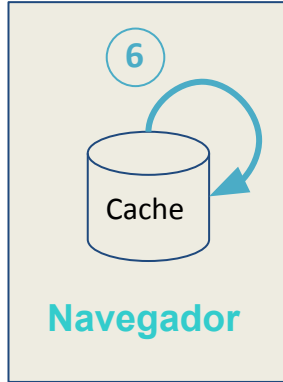
5) Servidor web retorna a página solicitada



# Funcionamento Básico do Web *Caching*



Usuário



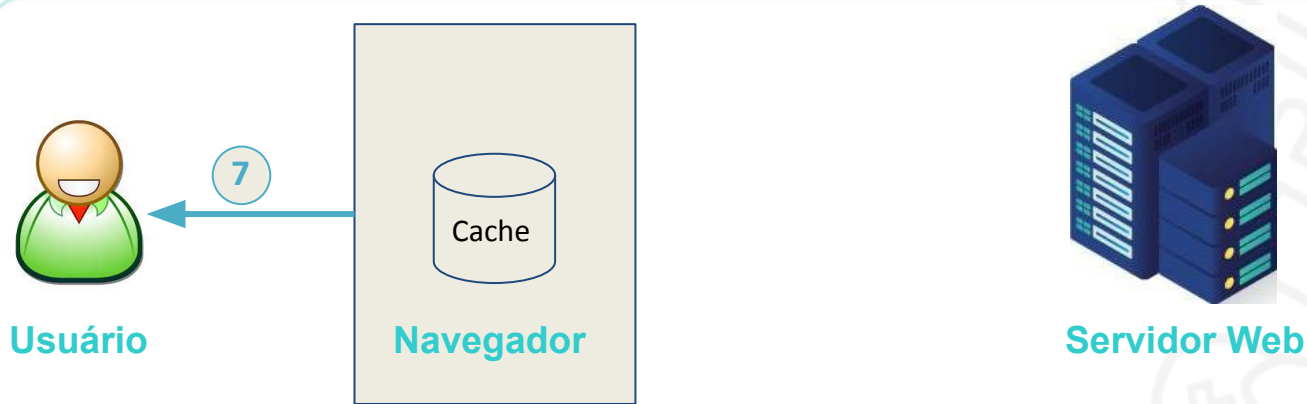
Navegador



Servidor Web

6) Navegador atualiza a página na *cache*

# Funcionamento Básico do Web *Caching*



7) Navegador retorna a página para o usuário

# Vantagens, Desvantagem e Desafio

- **Vantagens:** reutilização de páginas na *cache*, melhorando desempenho do serviço web e reduzindo tráfego e latência na rede
- **Desvantagem:** navegador precisa armazenar páginas, contudo, isso é barato
- **Desafio:** determinar se uma cópia em *cache* está atualizada

# Estratégias para Validar se Página está Atualizada

- **Primeira Estratégia:** *Cache* valida a página antes de informar o *cache hit*
- **Segunda Estratégia:** Navegador pergunta ao servidor web se sua cópia da página está atualizada

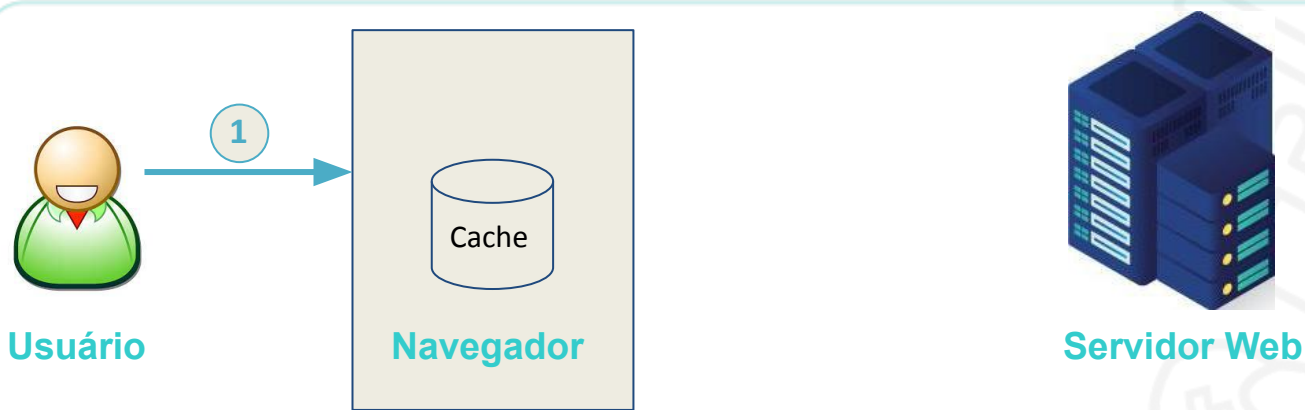
# Primeira Estratégia para Validar Página

- *Cache* usa o cabeçalho *Expires* da página e a data atual para verificar se a página está atualizada
- Entretanto, o *Expires* pode estar errado. Afinal, é difícil prever o futuro
- Podemos fazer inferências usando heurísticas, por exemplo:
  - Se o cabeçalho *Last-Modified* da página é antigo, apostamos que ela está atualizada.
  - Trabalhando com ações no domingo (sendo *Last-Modified* de sexta-feira), apostamos que a página está atualizada. No horário comercial, apostamos o contrário

# Segunda Estratégia para Validar Página

- A primeira estratégia é interessante, contudo, depende de uma aposta
- A segunda estratégia é perguntar ao servidor web se a cópia na *cache* é válida. Isso é feito através de um GET condicional
- Se verdadeiro, o servidor responde com uma resposta positiva curta. Caso contrário, ele envia a resposta completa

# Nova Versão do Funcionamento Básico

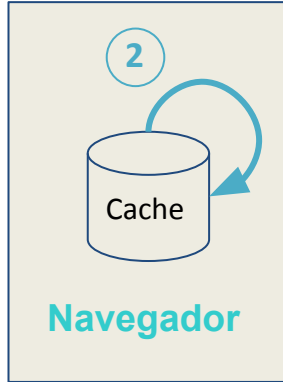


1) Usuário solicita página para o navegador

# Nova Versão do Funcionamento Básico



Usuário



Navegador

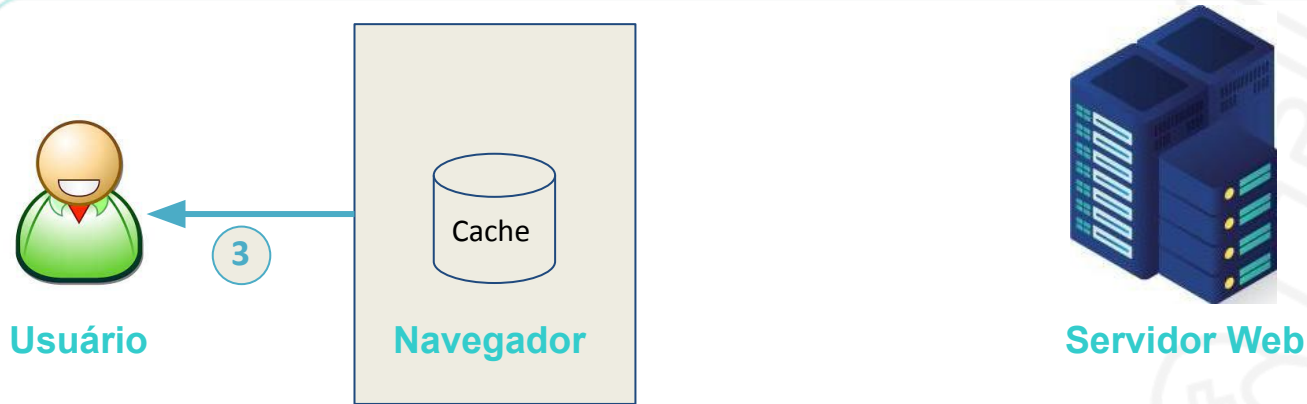


Servidor Web

2) Navegador verifica se existe a página **A.T.U.A.L.I.Z.A.D.A** na *cache*



# Nova Versão do Funcionamento Básico

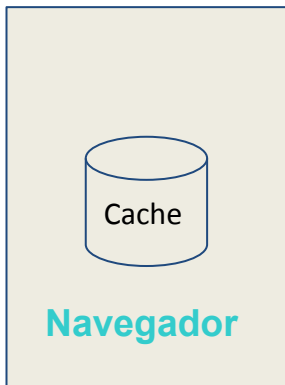


3) Se verdadeiro, o navegador retorna a página para o usuário sem acessar o servidor web nem gastar recursos de rede

# Nova Versão do Funcionamento Básico



Usuário



Navegador

4



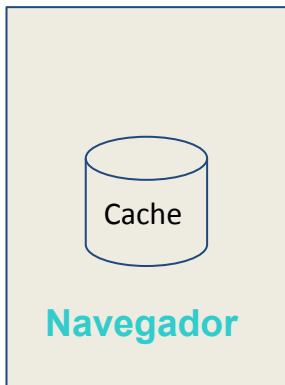
Servidor Web

- 4) Se não existir a página na *cache* ou navegador preferir confirmar a atualização com o servidor, ele envia uma GET condicional

# Nova Versão do Funcionamento Básico



Usuário



Navegador

5



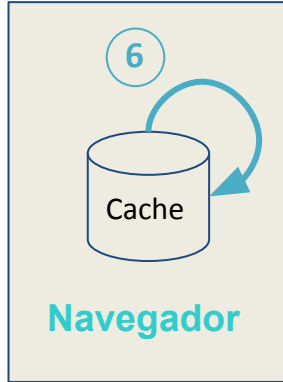
Servidor Web

- 5) Se a página estiver atualizada, o servidor envia uma resposta curta. Caso contrário, a página completa

# Nova Versão do Funcionamento Básico



Usuário



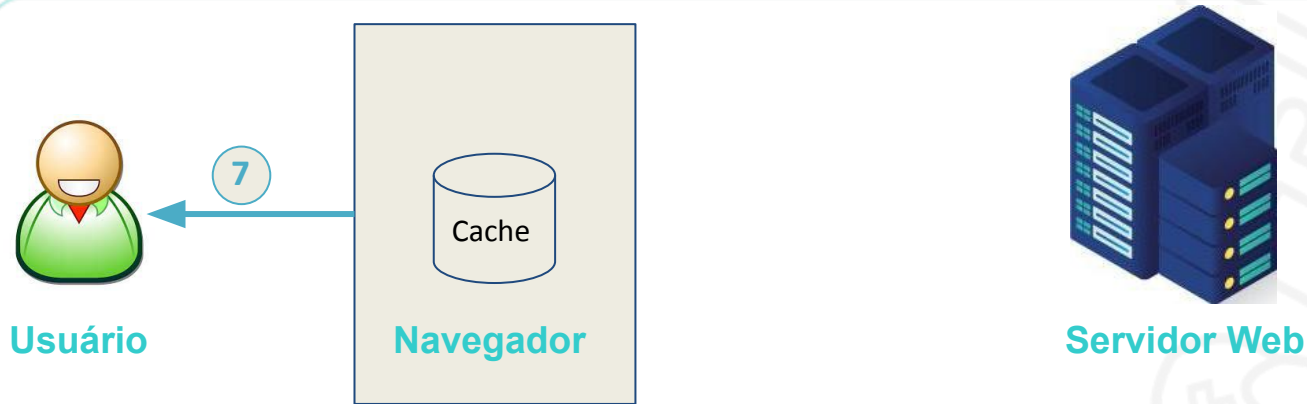
Navegador



Servidor Web

6) Navegador atualiza a *cache*

# Nova Versão do Funcionamento Básico



7) Navegador retorna a página para o usuário

# Considerações sobre *Web Caching*

- O cliente tem o cabeçalho *Last-Modified* e pode enviar um *If-Modified-Since* para solicitar uma página somente se sua cópia estiver desatualizada
- *Web Caching* é um tema interessante, amplo e que tem um efeito considerável sobre o desempenho
- *Proxies* também implementam *caching*, trazendo informações para mais perto da sua rede e reduzindo a latência e o tráfego de dados

# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - Melhorando Desempenho do HTTP
  - Versões
    - Caching
    - Replicação de Servidores
    - Redes de Entrega de Conteúdo

# Replicação de Servidores (*Mirroring*)

- Replicar o conteúdo de um servidor em múltiplas localidades separadas
- Funcionamento básico:
  - Página principal contém *links* para sites de algumas regiões
  - Usuário clica no *link* do site mais próximo e as demais solicitações vão para o servidor selecionado
  - O redirecionamento também pode ser automático



# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - Melhorando Desempenho do HTTP
  - Versões
    - Caching
    - Replicação de Servidores
    - Redes de Entrega de Conteúdo

# Redes de Entrega de Conteúdo

- Do inglês, *Content Delivery Networks* (CDN)
- Empresas que disseminam conteúdos de outras empresas para os usuários
- Replicam conteúdos, melhorando desempenho

# Redes de Entrega de Conteúdo

```
<html>
<head><title>Página Web</title></head>
<body>
 <h1>Lista de produtos</h1>
 Chocolate

 Hamburger

 Pão

</body>
</html>
```

(a) Página web original

```
<html>
<head><title>Página Web</title></head>
<body>
 <h1>Lista de produtos</h1>
 Chocolate

 Hamburger

 Pão

</body>
</html>
```

(b) A mesma página depois de ser transformada

# Passos ao se Carregar uma URL Utilizando CDN



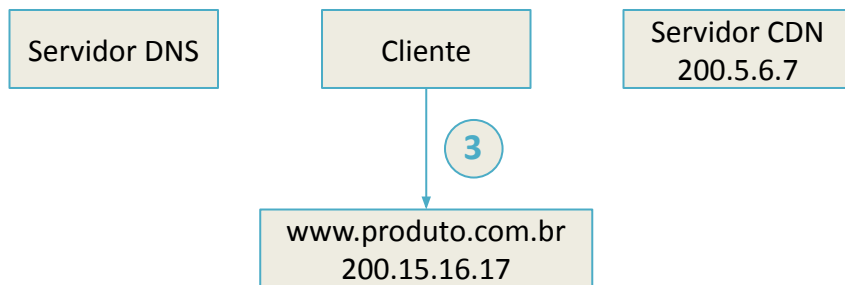
1. **Solicita endereço de [www.produtos.com.br](http://www.produtos.com.br)**
2. Envia endereço IP 200.15.16.17
3. Solicita page.html
4. Envia page.html
5. Solicita endereço de cdn.org
6. Envia endereço IP 200.5.6.7
7. Solicita chocolate.mpg
8. Envia chocolate.mpg
9. Salva chocolate.mpg no *cache*
10. Exibe chocolate.mpg

# Passos ao se Carregar uma URL Utilizando CDN



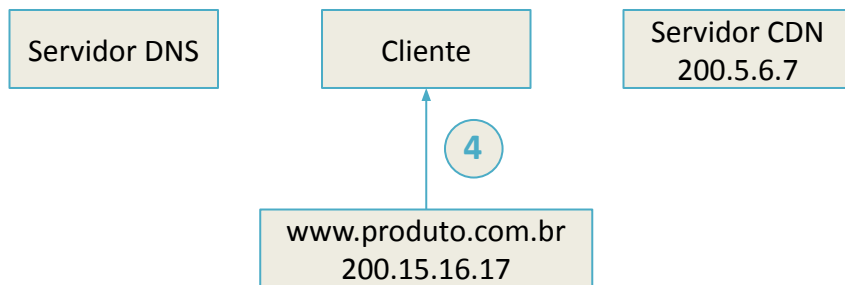
1. Solicita endereço de `www.produtos.com.br`
2. **Envia endereço IP 200.15.16.17**
3. Solicita `page.html`
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. Solicita `chocolate.mpg`
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. **Solicita `page.html`**
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. Solicita `chocolate.mpg`
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. Solicita `page.html`
4. **Envia `page.html`**
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. Solicita `chocolate.mpg`
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de [www.produtos.com.br](http://www.produtos.com.br)
2. Envia endereço IP 200.15.16.17
3. Solicita [page.html](#)
4. Envia [page.html](#)
5. **Solicita endereço de [cdn.org](http://cdn.org)**
6. Envia endereço IP 200.5.6.7
7. Solicita [chocolate.mpg](#)
8. Envia [chocolate.mpg](#)
9. Salva [chocolate.mpg](#) no *cache*
10. Exibe [chocolate.mpg](#)



# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. Solicita `page.html`
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. **Envia endereço IP `200.5.6.7`**
7. Solicita `chocolate.mpg`
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. Solicita `page.html`
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. **Solicita `chocolate.mpg`**
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. Solicita `page.html`
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. Solicita `chocolate.mpg`
8. **Envia `chocolate.mpg`**
9. Salva `chocolate.mpg` no *cache*
10. Exibe `chocolate.mpg`

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de www.produtos.com.br
2. Envia endereço IP 200.15.16.17
3. Solicita page.html
4. Envia page.html
5. Solicita endereço de cdn.org
6. Envia endereço IP 200.5.6.7
7. Solicita chocolate.mpg
8. Envia chocolate.mpg
9. **Salva chocolate.mpg no cache**
10. Exibe chocolate.mpg

# Passos ao se Carregar uma URL Utilizando CDN



1. Solicita endereço de `www.produtos.com.br`
2. Envia endereço IP `200.15.16.17`
3. Solicita `page.html`
4. Envia `page.html`
5. Solicita endereço de `cdn.org`
6. Envia endereço IP `200.5.6.7`
7. Solicita `chocolate.mpg`
8. Envia `chocolate.mpg`
9. Salva `chocolate.mpg` no *cache*
10. **Exibe `chocolate.mpg`**

# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - *Caching*
  - *Versões do HTTP*
  - HTTP vs. SMTP

# Protocolo de Transporte para o HTTP

- É o TCP para abstrair o tratamento mensagens longas, confiabilidade ou controle de congestionamento
- O HTTP/1.0 abre e fecha uma conexão TCP para tratar um único par solicitação/resposta porque as páginas eram basicamente texto HTML
- Contudo, as páginas evoluíram, ganharam muitos *links* inseridos para conteúdos como ícones e outros atrativos visuais, demandando a existência de uma conexão para cada atrativo visual

# HTTP/1.1

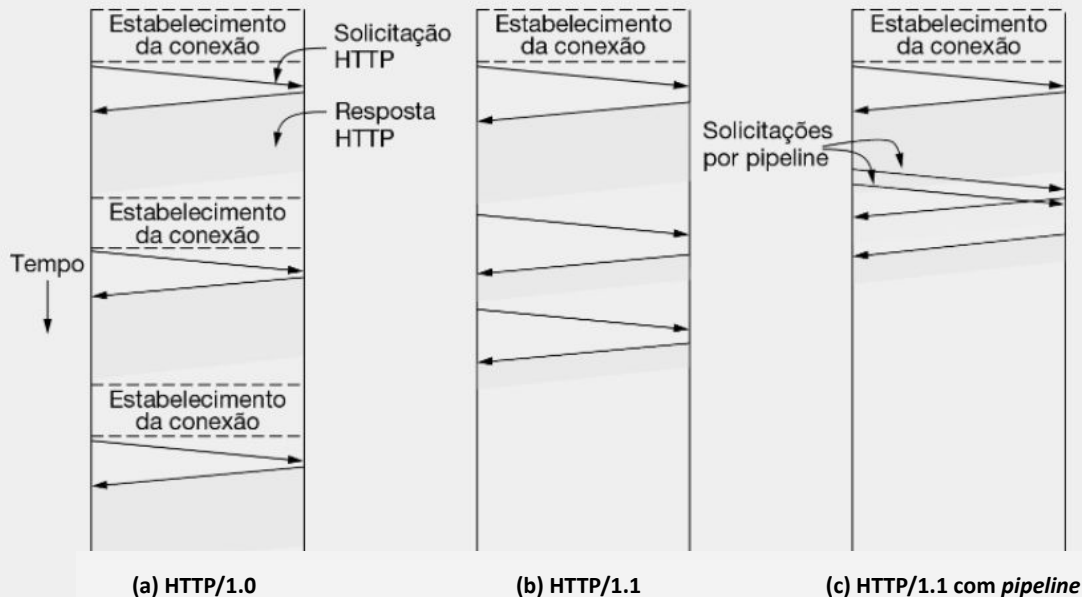
- Tem uma conexão persistente (reuso de conexão) que utiliza a mesma conexão em todas as solicitações/respostas relativas à uma mesma página
- Conexão persistente também permite o *pipeline* de solicitações, ou seja, enviar uma solicitação antes de chegar a resposta da solicitação anterior



# Exemplo das Conexões TCP no HTTP/1.0 e 1.1

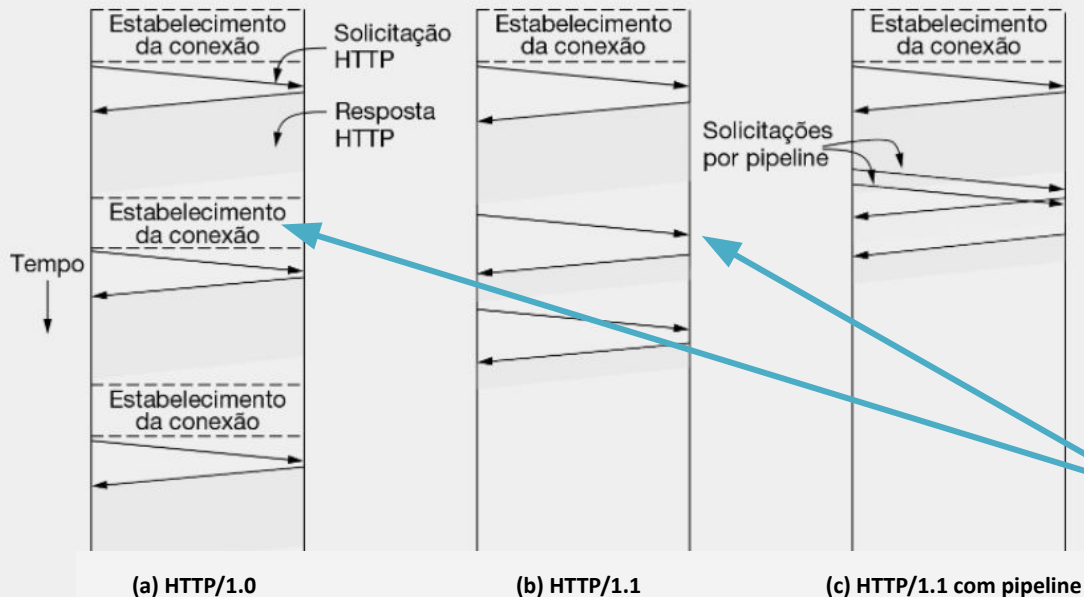
- Suponha que um cliente deseja uma página que tem duas imagens inseridas, sendo que a páginas e as imagens estão disponíveis no mesmo servidor
- Cliente só saberá da existência das imagens à medida que receber página
- Atualmente, uma página tem mais de 40 objetos que precisam ser buscados para apresentá-la

# Exemplo das Conexões TCP no HTTP/1.0 e 1.1



FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

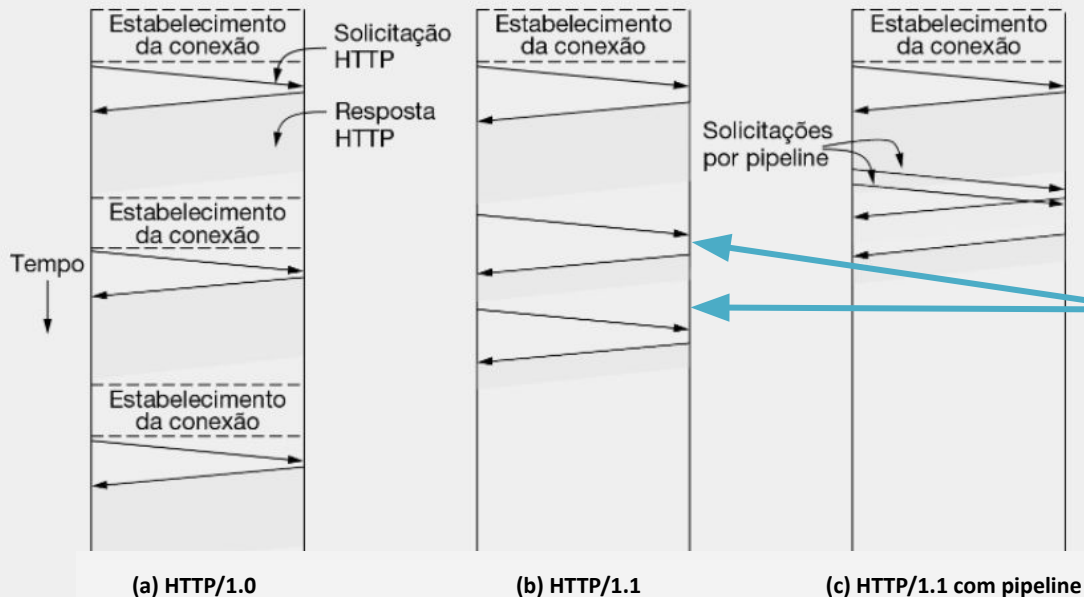
# Exemplo das Conexões TCP no HTTP/1.0 e 1.1



Conexão persistente elimina o custo de conexões adicionais

FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

# Exemplo das Conexões TCP no HTTP/1.0 e 1.1

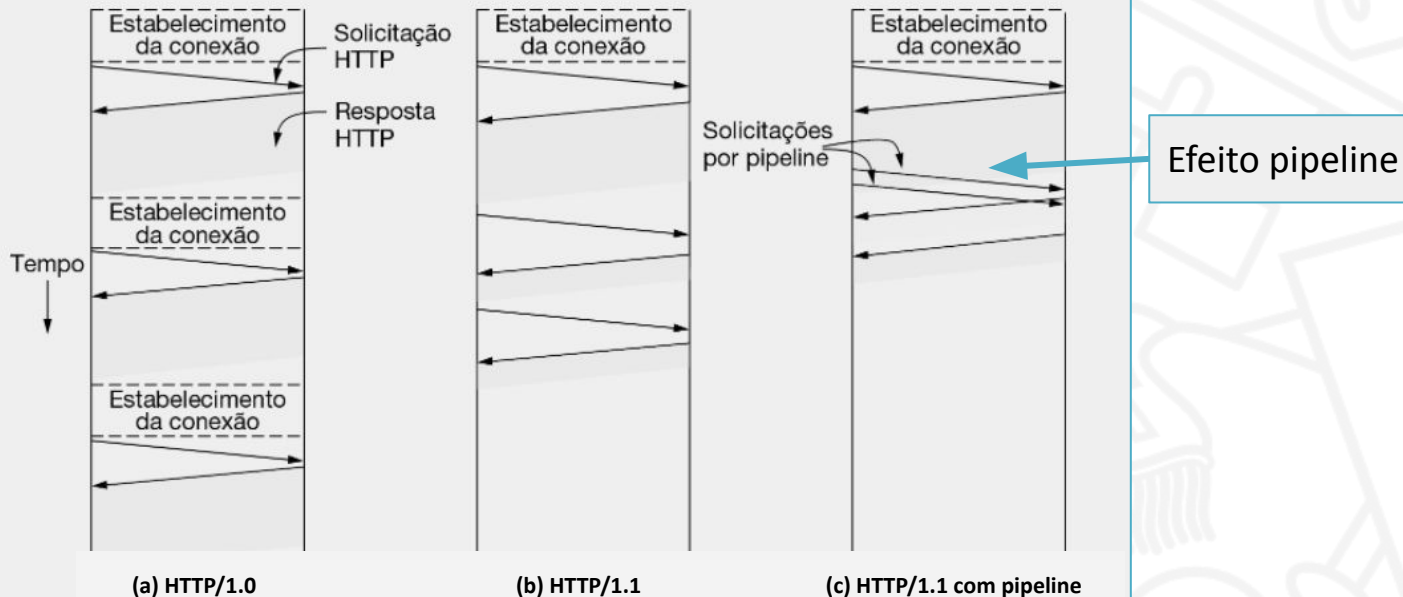


Controle de congestionamento do TCP acelera transferências adicionais

TCP usa um procedimento de partida lenta para aumentar a taxa de transferência até aprender o comportamento do caminho da rede. As demais transferências se aproveitam dessa informação

FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

# Exemplo das Conexões TCP no HTTP/1.0 e 1.1



FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

# Desafio das Conexões TCP

- Determinar quando fechar uma conexão, pois o usuário pode clicar em um *link* que solicita outra página no mesmo do servidor

**Resposta:** Manter uma conexão aberta se ela não ficar ociosa (e.g., por 60 segundos) ou se alcançarmos um número limite de conexões abertas

# Outras Melhorias no HTTP 1.1

- Servidor pode enviar uma resposta sem conhecer seu tamanho total (e.g., saída de um script) usando o cabeçalho *Transfer-Encoding: Chunked*
- Acrescentou suporte a *cache*
- Uso eficiente de endereços IP, permitindo que vários domínios sejam servidos pelo mesmo endereço IP
- Inclui o cabeçalho “Host:” em todas as requisições

# Histórico do HTTP

- HTTP/1.0, início da web
- HTTP/1.1, 2007
- HTTP/2, 2015 (RFC 7540)
- HTTP/3 (em breve)



# Melhorias no HTTP/2

- Permite a escolha da versão do HTTP
- Melhora o desempenho com multiplexação, *pipelining* e compactação
- Muda a codificação e o modo como clientes e servidores interagem
- No HTTP/1.1, o cliente abre a conexão TCP, envia sua solicitação **como texto**, espera uma resposta e fecha a conexão

# Solicitações e Respostas no HTTP/2

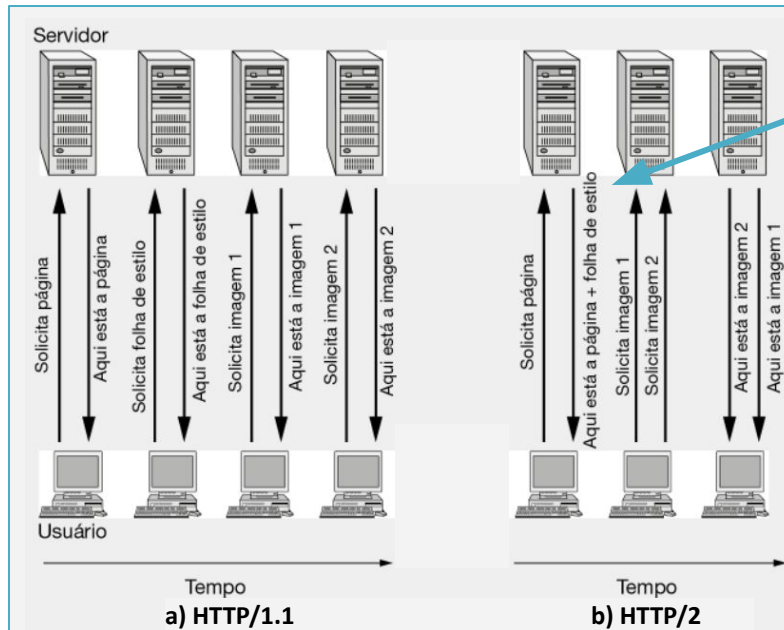
- Solicitações podem ser enviadas em binário e priorizadas
- Servidor responde às solicitações na ordem que desejar
  - Por exemplo, se o servidor tem que retornar 2 imagens e a primeira é maior, ele pode enviar a segunda, permitindo a visualização dessa antes da primeira ficar disponível
- Conexão é fechada somente quando todas as solicitações forem respondidas

# Server Push do HTTP/2

- Servidor envia arquivos que serão necessários e o cliente ainda não sabe
- Um exemplo acontece quando o cliente solicita uma página que tem uma folha de estilo e um arquivo JavaScript
- Outro exemplo acontece quando o cliente solicita uma página que tem uma folha de estilo e duas imagens (ilustração na próxima página)

## Exemplo *Server Push* do HTTP/2

- Outro exemplo acontece quando o cliente solicita uma página que tem uma folha de estilo e duas imagens



Servidor enviou a folha sem que o cliente pedisse

Servidor enviou a primeira imagem antes da segunda

FONTE: A. Tanenbaum et. al. Redes de Computadores. 6ª ed. 2021. Editora Pearson.

# Melhorias no HTTP/2

- O HTTP/2 também compacta os cabeçalhos e os envia em binário para reduzir a largura de banda e a latência
- Criptografia foi um ponto sensível durante o desenvolvimento do HTTP/2, pois algumas pessoas queriam por motivos óbvios e outras eram contra pela restrição computacional (em especial na Internet das Coisas)

# HTTP/3

- Utiliza o QUIC, um novo protocolo de transporte uma versão do UDP com controle de congestionamento
- Existem muitas bibliotecas (em C, C++, Python, Rust e GO) de código aberto e servidores web (e.g., nginx) que suportam QUIC e HTTP/3
- QUIC suporte especial para stream e executado exclusivamente com HTTPs
- QUIC e HTTP/3 têm otimização de conexões

# Agenda

- Introdução
  - Visão Geral da Arquitetura Web
  - Páginas Estáticas e Dinâmicas
  - *HyperText Transfer Protocol (HTTP)*
  - Privacidade na Web
- Visão Geral
  - Métodos
  - Cabeçalhos de Mensagens
  - Melhorando Desempenho do HTTP
  - Versões do HTTP
  - **HTTP vs. SMTP**

# Semelhanças entre HTTP e SMTP

- Usados para transferir arquivos de um hospedeiro para outro
- Utilizam conexões persistentes



# Diferenças entre HTTP e SMTP

HTTP	SMTP
● Protocolo de recuperação de informações    ● Resposta tem o cabeçalho Content-Length, permitindo ao cliente delinear o final da mensagem	● Protocolo de envio de informações    ● Mensagens no formato ASCII de 7 bits    ● Corpo da mensagem termina com CRLF.CRLF    ● Cada linha tem (no máximo) 1000 caracteres    ● Conversão de dados binários para ASCII (e.g., base64)
● Encapsula cada objeto em sua mensagem de resposta HTTP	● Coloca todos os objetos da mensagem em uma só mensagem

# Privacidade na Web

# Visão Geral

- Sites usam mecanismos HTTP para rastrear o comportamento do usuário
- Sites contratam redes de publicidade que obtêm acesso a informações sobre o comportamento dos usuários em cada um dos clientes (sites)
- Invasores rastreiam usuários a partir de informações disponíveis no navegador ou dispositivo dos usuários

# Cookies

- Forma convencional de implementar o rastreamento de usuários
- Importante para auxiliar um site em visitas futuras de um usuário
- Melhoram a experiência do usuário (e.g., sites personalizados, carrinhos de compra)
- Servidor utiliza o cabeçalho “set-cookie” do HTTP para inserir um pedaço de estado persistente em sua aplicação rodando no navegador



# Exemplo Positivo de Rastreamento

- Rastreamento entre dispositivos pode ser útil para prevenir fraudes
- Por exemplo, quando um provedor percebe que um usuário se conectou a partir de um dispositivo/local desconhecido, o provedor pode solicitar uma autenticação adicional (e.g., autenticação de dois fatores)

# Super Cookies

- Permitem intermediários rastreiem um usuário em sites ao longo do tempo
- Não são controlados pelo usuário como nos *cookies* normais
- Podem incluir identificadores de rastreamento que não são excluídos quando o usuário apaga seus cookies

# Política de Mesma Origem

- Determina que *cookies* definidos por um domínio sejam visíveis apenas para ele por questões de segurança e privacidade

# Identificação de Dispositivo e Navegador

- Pode acontecer até mesmo quando o usuário desabilita mecanismos comuns de rastreamento, como cookies de terceiros
- Baseada em informações ambientais, contextuais e do dispositivo
- Por exemplo, a técnica de identificação de tela utiliza o código HTML para identificar um dispositivo, considerando diferenças na renderização, suavização, dimensões e fontes específicas para cada dispositivo



# Identificação de Dispositivo e Navegador

- Outro exemplo acontece quando sites usam APIs do navegador para obter informações (e.g., status da bateria) que identificam um dispositivo
- Informações de rastreamento também podem ser combinadas, aumentando a qualidade das mesmas